

CLASE 4

BACKTRACKING

Elegir, explorar y volver atrás.

Taller de Programación Competitiva | 2026



UNA PALABRA ESCONDIDA

Problema motivador

Queremos formar la palabra **BACKTRACK** recorriendo celdas vecinas en cualquier dirección. Una celda no puede utilizarse dos veces en el mismo recorrido.

¿Cómo elegimos el siguiente paso sin saber todavía si funcionará?

X	A	K	P
B	C	T	N
A	R	K	R
V	E	A	C



PARTE 1 • RECURSIÓN

Una llamada puede dejar otra tarea pendiente.



UNA FUNCIÓN PUEDE LLAMARSE A SÍ MISMA

```
1 def countdown(n):  
2     if n == 0:  
3         print("Despegue")  
4         return  
5  
6     print(n)  
7     countdown(n - 1)
```

Caso base

Resuelve directamente el caso más pequeño y detiene las llamadas.

Paso recursivo

Reduce el problema y llama a la misma función con una entrada más cercana al caso base.

LAS LLAMADAS QUEDAN PENDIENTES

Al ejecutar `countdown(3)`:

$3 \longrightarrow 2 \longrightarrow 1 \longrightarrow 0$

Al llegar a 0, las llamadas regresan en el orden contrario.

La cantidad de llamadas pendientes es la **profundidad** de la recursión.

Sin caso base

La función seguiría llamándose. Python finalmente produce un `RecursionError`.

Costo de memoria

Cada llamada guarda sus parámetros y variables locales hasta que la llamada siguiente termina.



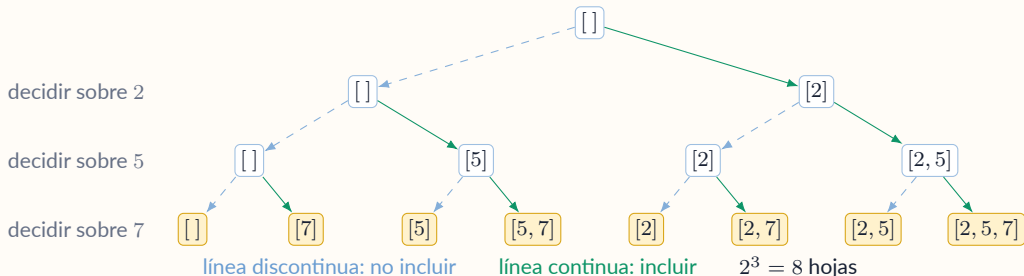
PARTE 2 • TODAS LAS POSIBILIDADES

Cada decisión abre una rama.



INCLUIR O NO INCLUIR

Queremos enumerar todos los subconjuntos de $[2, 5, 7]$.



ENUMERAR SUBCONJUNTOS

```
1 values = [2, 5, 7]
2 current = []
3 subsets = []
4
5 def search(index):
6     if index == len(values):
7         subsets.append(current.copy())
8         return
9
10    search(index + 1)
11
12    current.append(values[index])
13    search(index + 1)
14    current.pop()
15
16 search(0)
17 print(subsets)
```

append agrega y pop deshace ese cambio.

copy guarda una copia de la lista actual.

LAS CUATRO PIEZAS DEL BACKTRACKING

1 • Estado

`index` indica la próxima decisión.
`current` contiene exactamente los elementos elegidos en el camino actual.

2 • Decisiones

Para el elemento actual exploramos las ramas *no incluir* e *incluir*.

3 • Caso base

Cuando no quedan decisiones, el estado representa una respuesta completa.

4 • Deshacer

Antes de regresar dejamos `current` exactamente como estaba al entrar a la llamada.

AVANZAR Y VOLVER ATRÁS

Para explorar la rama que incluye 5 hacemos

$$[2] \xrightarrow{\text{append}(5)} [2, 5].$$

Cuando esa llamada termina, debemos recuperar el estado anterior:

$$[2, 5] \xrightarrow{\text{pop}()} [2].$$

Invariante

Al comenzar y al terminar cada llamada recursiva, `current` contiene las elecciones realizadas antes de `index`.

BACKTRACK

significa regresar por el camino sin dejar cambios olvidados.

¿POR QUÉ APARECEN TODOS LOS SUBCONJUNTOS?

No falta ninguno

Todo subconjunto determina una secuencia única de decisiones. Para cada elemento dice si se incluye o no. El algoritmo recorre esa secuencia.

No se repite ninguno

Dos secuencias distintas difieren en alguna decisión. Por lo tanto, producen subconjuntos distintos.

Las hojas del árbol están en correspondencia uno a uno con las respuestas.

EL ÁRBOL PUEDE CRECER MUY RÁPIDO

Con n elementos tenemos 2^n subconjuntos.

Tiempo

Guardar una copia de cada subconjunto cuesta

$$\mathcal{O}(n 2^n).$$

El tamaño total de la salida puede alcanzar esa misma magnitud.

Memoria de la búsqueda

La lista actual y las llamadas pendientes ocupan $\mathcal{O}(n)$, sin contar la lista donde guardamos todas las respuestas.

Para $n = 40$, ya existen más de un billón de subconjuntos.

LAS DECISIONES NO SIEMPRE SON BINARIAS

Recordatorio de números primos

La factorización

$$360 = 2^3 \cdot 3^2 \cdot 5^1$$

implica que todo divisor positivo tiene la forma $2^a 3^b 5^c$.

Primo	Decisión	Opciones
2	elegir a	0, 1, 2, 3
3	elegir b	0, 1, 2
5	elegir c	0, 1

Un camino completo

Las decisiones $(a, b, c) = (2, 1, 0)$ producen $2^2 \cdot 3^1 \cdot 5^0 = 12$. Hay $(3 + 1)(2 + 1)(1 + 1) = 24$ hojas, una por divisor.

ENUMERAR LOS DIVISORES

```
1 factors = [(2, 3), (3, 2), (5, 1)]
2 divisors = []
3
4 def search(index, current):
5     if index == len(factors):
6         divisors.append(current)
7         return
8
9     prime, max_exponent = factors[index]
10    power = 1
11    for exponent in range(max_exponent + 1):
12        search(index + 1, current * power)
13        power *= prime
14
15 search(0, 1)
16 print(divisors)
```

Cada par guarda un primo y su exponente máximo.

`range(m + 1)` recorre desde 0 hasta m .

Estado inmutable

`current * power` crea un entero nuevo. La llamada anterior conserva su propio valor.

UNA HOJA POR DIVISOR

No falta ninguno

Cada divisor elige un exponente permitido para cada primo. La búsqueda recorre exactamente esa secuencia de decisiones.

No se repite ninguno

La factorización prima es única. Dos secuencias de exponentes distintas no pueden producir el mismo divisor.

Si hay k primos y D divisores, la búsqueda tarda $\mathcal{O}(D)$, usa $\mathcal{O}(k)$ memoria auxiliar y almacena $\mathcal{O}(D)$ respuestas.



PARTE 3 • RESTRICCIONES Y PODA

No recorramos caminos que ya sabemos que fallan.



VOLVAMOS A UNA RAMA CONCRETA

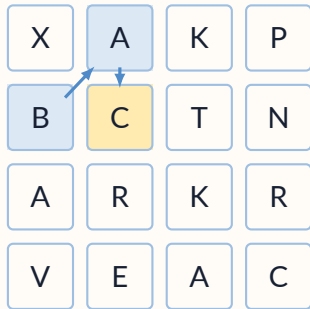
Ya elegimos tres celdas vecinas y formamos

$B \longrightarrow A \longrightarrow C.$

Ahora necesitamos la letra **K**.

Pregunta

¿Debemos continuar desde cada vecino de la C?



azul: camino anterior

amarillo: celda actual

DE OCHO OPCIONES QUEDAN DOS RAMAS

X	A	K
B	C	T
A	R	K

Continuar

Dos vecinas contienen K y todavía no fueron usadas.

Descartar

Dos pertenecen al camino actual y cuatro contienen otra letra.

Poda: detener una rama apenas sabemos que no puede funcionar.

¿QUÉ SIGNIFICA UNA LLAMADA?

Pregunta

`search(row, col, index)`
pregunta si podemos formar la palabra desde esa letra y esa celda.

Caso base

Después de validar la celda, encontrar la última letra completa la palabra.

Antes de continuar

La celda debe estar dentro de la grilla, no haber sido usada y contener la letra esperada.

Invariante

Las celdas marcadas con `None` son exactamente el camino anterior a la celda actual.

DATOS Y MOVIMIENTOS

```
1 board = [  
2     list("XAKP"),  
3     list("BCTN"),  
4     list("ARKR"),  
5     list("VEAC"),  
6 ]  
7 word = "BACKTRACK"  
8 rows = len(board)  
9 cols = len(board[0])  
10  
11 directions = [  
12     (-1, -1), (-1, 0), (-1, 1),  
13     ( 0, -1),      ( 0, 1),  
14     ( 1, -1), ( 1, 0), ( 1, 1),  
15 ]
```

`list("XAKP")` crea una lista que podemos modificar.

Cada par (`dr`, `dc`) cambia la fila y la columna para llegar a una de las ocho vecinas.

DESCARTAR ANTES DE RAMIFICAR

```
1 def search(row, col, index):
2     inside = 0 <= row < rows and 0 <= col < cols
3     if not inside:
4         return False
5
6     if board[row][col] is None:
7         return False
8     if board[row][col] != word[index]:
9         return False
10
11     if index == len(word) - 1:
12         return True
```

Tres podas

Retornamos si la celda está fuera, ya fue usada o contiene otra letra.

Orden importante

Revisamos los límites antes de acceder a `board[row][col]`.

EXPLORAR Y RESTAURAR

```
1 letter = board[row][col]
2 board[row][col] = None
3
4 found = False
5 for dr, dc in directions:
6     if search(row + dr, col + dc, index + 1):
7         found = True
8         break
9
10 board[row][col] = letter
11 return found
```

Cada par se separa en dr y dc. Así probamos las ocho decisiones posibles.

Error frecuente

Retornar verdadero dentro del ciclo dejaría la celda marcada. Primero restauramos y después retornamos.

PROBAR TODOS LOS COMIENZOS

```
1 def contains_word():
2     if len(word) > rows * cols:
3         return False
4
5     for row in range(rows):
6         for col in range(cols):
7             if search(row, col, 0):
8                 return True
9     return False
10
11 print(contains_word())
```

¿Dónde comienza?

Como no lo sabemos, probamos cada celda. Casi todas se descartan al no contener B.

Para nuestra grilla, el programa imprime

True.

¿POR QUÉ LA BÚSQUEDA ES CORRECTA?

Si responde verdadero

Cada llamada verificó la letra esperada. Las llamadas consecutivas usan celdas vecinas y None impide repetir una celda del camino.

Si existe un camino

Probamos su primera celda como comienzo. Después, en cada paso probamos las ocho vecinas. La siguiente celda de ese camino nunca será podada.

Restaurar la grilla garantiza que explorar una rama no modifica las demás.

EL COSTO SIGUE SIENDO EXPONENCIAL

Sea $R \times C$ el tamaño de la grilla y L el largo de la palabra.

Cota superior

Hay RC comienzos y hasta ocho decisiones en cada uno de los $L - 1$ pasos siguientes:

$$\mathcal{O}(RC \cdot 8^{L-1}).$$

Memoria

La profundidad nunca supera L . Como marcamos sobre la misma grilla, la búsqueda usa $\mathcal{O}(L)$ memoria adicional.

Poda antes de buscar

Si $L > RC$, formar la palabra exigiría repetir alguna celda. Podemos responder falso inmediatamente.

BACKTRACKING TAMBIÉN PUEDE OPTIMIZAR

Supongamos que cada celda tiene un valor. Entre todos los caminos que forman **BACKTRACK**, queremos encontrar el de mayor suma.

El estado no cambia

Seguimos guardando la celda actual, la próxima letra y las celdas del camino. También conservamos las mismas podas.

Cambia lo que retorna

Una rama inválida retorna $-\infty$.
Una rama válida suma el valor de su celda y el mejor resultado entre todas sus continuaciones.

Existir, contar y optimizar son recorridos distintos sobre el mismo árbol de decisiones.



PARTE 4 • N REINAS

Un buen estado evita trabajo antes de comenzar.



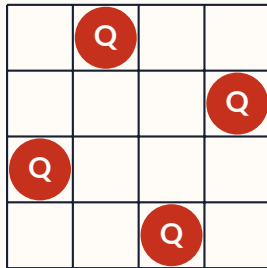
CUATRO REINAS QUE NO SE ATAQUEN

En un tablero de $N \times N$ queremos colocar N reinas.

Ningún par puede compartir fila, columna o diagonal.

Pregunta

¿Qué decisión debería representar un nivel del árbol?



UNA REINA POR FILA

Toda solución tiene exactamente una reina en cada fila.

Estado

row indica la fila que estamos completando. También guardamos las columnas y diagonales ya ocupadas.

Decisión

Elegir la columna de la reina en esa fila. Nunca consideramos colocar dos reinas en la misma fila.

Elegir bien el estado elimina desde el comienzo muchas configuraciones redundantes.

RECONOCER COLUMNAS Y DIAGONALES

Una celda tiene coordenadas (r, c) .

Columna

Dos celdas comparten columna cuando tienen el mismo valor de c .

Diagonal $r - c$

Comparten esta diagonal cuando tienen el mismo valor de $r - c$.

Diagonal $r + c$

Comparten esta diagonal cuando tienen el mismo valor de $r + c$.

Usamos arreglos booleanos. La columna ocupa la posición c , mientras que las diagonales ocupan $r - c + N - 1$ y $r + c$. Los tres índices son no negativos.

CONTAR SOLUCIONES DE N REINAS, PARTE 1 DE 2

```
1 n = 8
2 used_column = [False] * n
3 used_down = [False] * (2 * n - 1)
4 used_up = [False] * (2 * n - 1)
5
6 def search(row):
7     if row == n:
8         return 1
9
10    total = 0
11    for col in range(n):
12        down = row - col + n - 1
13        up = row + col
14        if (used_column[col] or used_down[down]
15            or used_up[up]):
16            continue
```

CONTAR SOLUCIONES DE N REINAS, PARTE 2 DE 2

```
1      used_column[col] = True
2      used_down[down] = True
3      used_up[up] = True
4
5      total += search(row + 1)
6
7      used_column[col] = False
8      used_down[down] = False
9      used_up[up] = False
10     return total
11
12     print(search(0))
```

TIEMPO $\mathcal{O}(N!)$

MEMORIA $\mathcal{O}(N)$

LA PODA CONSERVA EXACTAMENTE LAS SOLUCIONES

Toda hoja contada es válida

Colocamos una reina por fila.
Los tres arreglos impiden
compartir columna o diagonal.
Al llegar a la fila N tenemos
una solución.

Toda solución aparece

En cada fila probamos toda
columna que no ataca a las
anteriores. La columna usada
por cualquier solución válida
nunca será podada.

Sin considerar diagonales, quedan a lo sumo $N!$ formas de elegir columnas. Las diagonales reducen todavía más el árbol.

UNA RECETA PARA DISEÑAR BACKTRACKING

1. Definir qué significa el estado.
2. Elegir cuál es la próxima decisión.
3. Reconocer cuándo una respuesta está completa.
4. Probar cada opción posible.
5. Podar apenas una rama sea imposible.
6. Deshacer todos los cambios antes de regresar.

Ojo

Backtracking suele ser exponencial, así que los límites deben ser pequeños. Muchas veces la cota superior asusta y confiamos en que las podas reduzcan muchísimo el árbol.

LO QUE DEBEMOS RECORDAR

Recursión

Un caso base detiene las llamadas. La profundidad determina la memoria de la búsqueda.

Backtracking

El estado describe el camino actual. Elegimos, exploramos y deshacemos cada cambio.

Poda

Descartamos una rama solamente cuando podemos justificar que no contiene respuestas.